

ACADEMY

Pulso TransMi

Guía metodológica del proyecto MLOps · versión 1.0 · 16 de septiembre de 2026

DOCUMENTO	CURSO	FECHA	USO
Material académico	MLOps · Ciencia de Datos	16 de septiembre de 2026	Uso académico

INSTITUCIÓN	UNIDAD ACADÉMICA	DOCENTE
Universidad Externado de Colombia	Departamento de Matemáticas	Julián Zuluaga · Docente

Resumen ejecutivo

Pulso TransMi es un reto para aprender a **operar machine learning en el tiempo**. El problema no consiste únicamente en entrenar un modelo y presentar una métrica. Cada equipo construirá un sistema capaz de recibir datos nuevos, producir pronósticos, observar sus errores y reaccionar cuando el comportamiento de la ciudad cambie.

Conclusión. El proyecto se resuelve construyendo un ciclo operativo completo, no persiguiendo una única métrica de laboratorio.

Impacto. Cada predicción deja evidencia para evaluar el modelo y decidir si debe mantenerse, investigarse o reemplazarse.

Acción del equipo. Comenzar por una memoria de datos confiable y avanzar hacia automatización, monitoreo y reentrenamiento.

La idea central

Un modelo útil hoy puede dejar de serlo mañana. El proyecto se evalúa por la capacidad del equipo para sostener un ciclo confiable de datos, modelos, predicciones, evaluación y mejora.

El reto en una frase

Pronosticar la demanda futura de pasajeros en estaciones de TransMilenio mientras los patrones cambian y obligan a revisar, comparar y reentrenar modelos.

Qué aprenderás

- A convertir una fuente de datos en un proceso automatizado y trazable.
- A separar la experimentación del modelo que realmente opera.
- A tomar decisiones de promoción y reemplazo con evidencia.
- A medir accuracy a lo largo del tiempo, no en una sola partición.
- A reconocer drift y diseñar una respuesta razonable.
- A coordinar datos, cómputo y visualización usando servicios gratuitos.

La ciudad como flujo de datos



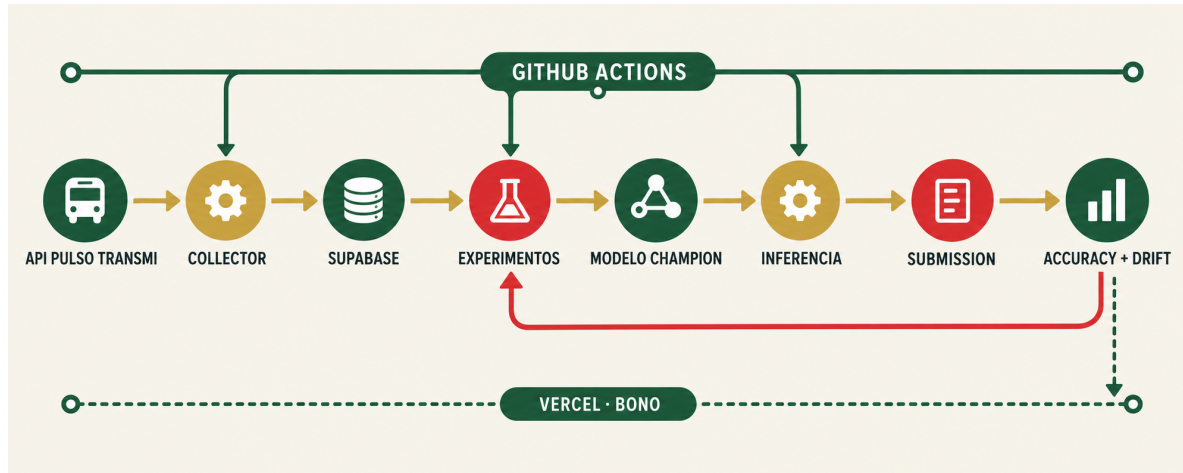
La movilidad genera observaciones; el sistema aprende de ellas y devuelve pronósticos que vuelven a ponerse a prueba.

Fuente: Elaboración editorial con IA para Pulso TransMi, 2026-09-16

Cada estación produce una historia temporal. El reto consiste en convertir esas señales dispersas en una predicción trazable y volver a contrastarla con la ciudad cuando aparezca el dato real.

El sistema completo

El proyecto tiene cuatro capas que trabajan juntas. La **API Pulso TransMi** representa el mundo observado; **Supabase** conserva la memoria del equipo; **GitHub Actions** ejecuta el trabajo periódico; y **Vercel**, cuando se implemente el bono, convierte el estado del sistema en una experiencia visible.



Arquitectura conceptual. El flujo produce predicciones y el monitoreo devuelve evidencia al espacio de experimentación.

Fuente: Elaboración editorial a partir del esquema de clase, 2026-09-16

La flecha de regreso es tan importante como el flujo hacia adelante. Cuando el accuracy cae o los datos se alejan de lo aprendido, el equipo vuelve a experimentar. Ese retorno convierte un ejercicio de modelado en un sistema MLOps.

Principio de responsabilidad

Capa	Pregunta que responde
Datos	¿Qué sabemos hasta este momento?
Modelo	¿Qué patrón creemos que continuará?
Operación	¿Cuándo y con qué versión se produjo cada predicción?
Monitoreo	¿El modelo sigue siendo suficientemente bueno?

Las plataformas y su papel

API Pulso TransMi — la realidad compartida

Es la fuente oficial del reto. Ya publica el histórico inicial, las estaciones y el contexto. Durante la competencia expondrá el reloj, liberará observaciones incrementales, identificará el ciclo vigente, recibirá submissions y actualizará el leaderboard.

La API es común para todos; lo que diferencia a los equipos es la manera en que construyen su memoria, sus modelos y sus decisiones.

Supabase — la memoria operacional

Cada equipo crea un proyecto de Supabase con PostgreSQL. Allí deben vivir, de manera ordenada:

- las estaciones y observaciones descargadas;
- el contexto disponible al momento de predecir;
- el historial de ejecuciones del collector;
- las versiones promovidas de modelos;
- las predicciones emitidas;
- el resultado real cuando se conozca;
- las métricas de accuracy y drift.

Supabase Storage funciona como bodega de artefactos: cada modelo promovido se guarda como una versión identificable, sin sobrescribir silenciosamente el anterior.

GitHub — el cuaderno reproducible

El repositorio contiene el código, las decisiones, el esquema de datos y la evidencia necesaria para volver a ejecutar el proyecto. Un resultado que solo funciona en el computador de una persona no está terminado.

GitHub Actions — el operador automático

Actions será el reloj del equipo. Ejecutará tareas como recolectar datos, entrenar, cargar el modelo promovido, predecir y evaluar. Su propósito no es ocultar el proceso, sino volverlo repetible y observable.

Vercel — la ventana del sistema

Vercel no entrenará modelos ni reemplazará GitHub Actions. Su función es publicar el dashboard opcional para que cualquier persona pueda entender el estado del proyecto.

Dos momentos distintos del proyecto

Desde ahora — construir y ensayar

La API ya ofrece un histórico fijo de **45 días**, **12 estaciones** y **51.840 observaciones** con granularidad de 15 minutos. Durante esta etapa el reloj de competencia está en `waiting`, el stream incremental está vacío y no hay ciclos abiertos. Esto significa que los equipos pueden diseñar y probar su arquitectura sin que aparezcan datos nuevos todavía.

El trabajo de esta semana es dejar preparado el sistema: repositorio, Supabase, migración del histórico, collector idempotente, análisis exploratorio, baselines, primer modelo champion y workflows de prueba.

La próxima semana — operar con datos nuevos

Cuando el escenario sea activado, la API comenzará a liberar observaciones en lotes. Desde ese momento cada equipo debe ejecutar su ciclo automáticamente: recolectar, consultar si existe un ciclo abierto, generar las predicciones solicitadas, enviarlas antes del cierre y evaluar su desempeño cuando aparezca la realidad.

Cómo saber si la competencia empezó

El equipo no debe adivinarlo por la fecha ni por la hora de su computador. El endpoint del reloj es la autoridad: mientras responda `waiting`, no se publican datos dinámicos ni se esperan submissions.

El ciclo MLOps



El aprendizaje no termina con el entrenamiento: cada resultado real alimenta la siguiente decisión.

Fuente: Elaboración editorial para Pulso TransMi, 2026-09-16

Observar

El equipo estudia los datos disponibles y formula hipótesis sobre horarios, estaciones, días, eventos y señales externas. Antes de elegir un algoritmo debe entender qué comportamiento intenta representar.

Entrenar y validar

Los experimentos usan particiones temporales. Mezclar aleatoriamente futuro y pasado produce resultados optimistas y no representa la competencia. Cada candidato debe compararse contra baselines simples.

Promover

Entrenar no equivale a desplegar. Promover significa declarar que una versión concreta es el **modelo champion** porque pasó los controles acordados y puede entrar al pipeline de inferencia.

Predecir y evaluar

El champion genera pronósticos para estaciones y horizontes futuros. Cuando se publica la observación real, el equipo une predicción y realidad, calcula la métrica y conserva la evaluación.

Detectar drift y reentrenar

Una caída persistente puede indicar que el patrón cambió. El equipo decide si hay suficientes datos nuevos, reentrena un candidato y solo reemplaza al champion cuando existe evidencia de mejora.

El reloj operativo de la competencia



Un solo envío cubre 48 targets. Los cuatro horizontes son momentos futuros a predecir, no cuatro submissions separadas.

Fuente: Elaboración editorial con IA a partir del contrato operativo Pulso TransMi v0.3.0, 2026-09-16

Todas las predicciones del ciclo se calculan y entregan juntas al comienzo. Los puntos **+15**, **+30**, **+45** y **+60** representan los momentos futuros que se quieren pronosticar; **no son cuatro envíos separados**.

Tres ritmos que no deben confundirse

Ritmo	Frecuencia	Qué significa para el equipo
Granularidad del dato	15 minutos	Cada observación representa un cuarto de hora por estación
Publicación de datos	30 minutos	La API libera dos timestamps nuevos por estación en cada lote
Ciclo de predicción	60 minutos	Se abre una oportunidad para enviar los cuatro horizontes

Cada ciclo permanece abierto durante **25 minutos reales**. Después del cierre no se aceptan entregas tardías. Los workflows gratuitos pueden sufrir retrasos, por lo que GitHub Actions solo despierta el proceso: la decisión final siempre se toma consultando el estado que devuelve la API.

Una rutina sugerida

- Cerca del minuto **07** : ejecutar collector, descubrir el ciclo, inferir y enviar si está abierto.
- Cerca del minuto **37** : recoger el siguiente lote, actualizar evaluaciones y revisar señales de degradación.
- En cada ejecución: continuar desde el cursor confirmado en Supabase y terminar correctamente si no hay novedades.
- En un workflow separado: entrenar por decisión o programación; no es obligatorio reentrenar cada 30 minutos.

Los minutos son una recomendación operacional, no una fuente de verdad. El cron de GitHub se configura en UTC y puede retrasarse; por eso ningún equipo debe fabricar un `cycle_id` a partir de la hora local.

Qué deben predecir exactamente

Un ciclo normal solicita **48 valores**:

12 estaciones × 4 horizontes = 48 predicciones

Los horizontes son demanda de pasajeros para +15 , +30 , +45 y +60 minutos respecto al origen del ciclo. La respuesta de ciclo vigente enumera cada pareja exacta de station_id y target_at ; esa lista es el contrato de la entrega.

Reglas conceptuales del batch

- 1. Usar únicamente información disponible hasta data_cutoff .
- 2. Generar una predicción finita y no negativa para cada target solicitado.
- 3. No agregar estaciones, timestamps ni horizontes que el ciclo no haya pedido.
- 4. Enviar el batch completo antes de closes_at .
- 5. Registrar la versión del modelo, el commit y el corte de entrenamiento.

La API acepta el batch completo o lo rechaza completo. Esta regla evita rankings contruirdos con coberturas distintas y hace que un error de ensamblaje sea visible en vez de quedar silencioso.

La escala del trabajo

Elemento	Valor oficial
Estaciones evaluadas	12
Horizontes por estación	4
Predicciones por ciclo	48
Ciclos ordinarios	1 por hora
Ventana de entrega	25 minutos
Intentos válidos máximos	3 por ciclo

Cómo se entrega una predicción

Antes de inferir

El workflow consulta el reloj y el ciclo vigente. Si no existe un ciclo abierto, termina sin error. Si existe, usa el `data_cutoff`, los targets y el deadline devueltos por la API; no los calcula por su cuenta.

Al enviar

Cada submission relaciona cuatro tipos de evidencia:

Evidencia	Para qué sirve
Identidad del participante	Se deriva de la API key; no se escribe el nombre en el payload
Identidad del ciclo	Evita que una predicción llegue al periodo equivocado
Trazabilidad del modelo	Versión, entrenamiento, corte de datos y commit
Predicciones	Parejas exactas de estación, instante objetivo y valor

La API key vive en GitHub Actions Secrets. No se guarda en el repositorio, en tablas accesibles desde el dashboard ni en variables públicas de Vercel.

Reintentos sin duplicados

Cada ejecución usa una llave de idempotencia estable. Si la red falla después de enviar, el workflow puede repetir la misma petición y recibir el mismo recibo sin crear otra entrega. Reutilizar esa llave con contenido diferente se considera un conflicto.

Cada equipo dispone de hasta **tres intentos válidos** por ciclo. El último aceptado antes del cierre reemplaza al anterior como entrega oficial; los intentos anteriores permanecen como evidencia. Guardar el `submission_id` y el recibo es parte del trabajo del pipeline.

Entrega aceptada no significa predicción correcta

La aceptación solo confirma que llegó a tiempo, con identidad válida y estructura completa. El desempeño se conoce cuando la API publica los valores reales de los targets.

Ruta metodológica

Fase 1 — Comprender

Pregunta: ¿qué hace difícil este pronóstico?

El equipo explora estacionalidades, diferencias entre estaciones, valores faltantes, anomalías y relaciones con el contexto. Debe entregar hipótesis, no únicamente gráficos.

Evidencia esperada: análisis exploratorio, diccionario básico de datos y definición clara de la variable objetivo.

Fase 2 — Construir la memoria

Pregunta: ¿podemos recuperar de forma confiable todo lo que el sistema ha observado?

Se crea la base en Supabase, se migra el histórico y se implementa un collector incremental. Ejecutarlo dos veces no debe duplicar información.

Evidencia esperada: esquema reproducible, datos cargados, bitácora de ingestiones y controles de calidad.

Fase 3 — Experimentar

Pregunta: ¿qué enfoque generaliza mejor hacia adelante?

Se construyen baselines y candidatos. Cada experimento conserva su ventana de entrenamiento, variables, parámetros y resultado de validación temporal.

Evidencia esperada: comparación razonada y selección de un candidato promovible.

Fase 4 — Operar

Pregunta: ¿el sistema puede actuar sin depender de una ejecución manual local?

GitHub Actions carga el champion, genera las predicciones requeridas, las registra y las envía antes del cierre del ciclo.

Evidencia esperada: workflow exitoso, batch identificable y submission aceptada.

Fase 5 — Aprender del error

Pregunta: ¿qué hacemos cuando el modelo deja de funcionar igual?

El equipo calcula accuracy móvil, identifica degradación y documenta su decisión: mantener, investigar o reentrenar.

Evidencia esperada: historial de desempeño, señal de drift y al menos una decisión sustentada.

El modelo promovido

El pipeline nunca debe cargar “el archivo más reciente” sin contexto. Cada modelo promovido necesita una identidad que permita explicar de dónde salió y reproducirlo.

La ficha mínima de una versión

Elemento	Qué permite responder
Identificador de versión	¿Cuál modelo produjo esta predicción?
Fecha y corte de datos	¿Qué información conocía al entrenarse?
Commit de código	¿Con qué implementación se construyó?
Features	¿Qué señales esperaba recibir?
Métrica de validación	¿Por qué fue considerado promovible?
Ubicación del artefacto	¿Cómo puede volver a cargarse?
Estado	¿Es candidato, champion o histórico?

Regla de promoción

Una nueva versión reemplaza al champion únicamente si supera los criterios de validación y puede completar una inferencia de prueba. La novedad por sí sola no es una mejora.

MLflow como bono de madurez

MLflow aporta una interfaz especializada para registrar experimentos, métricas, artefactos, versiones y linaje. Su uso suma valor si ayuda a responder mejor qué se entrenó, con cuáles datos y por qué se promovió. No reemplaza la obligación de tener un pipeline reproducible ni de conservar el modelo operativo.

Predicción, accuracy y drift

Una predicción es una promesa fechada

Una predicción válida debe existir antes de que se conozca la observación objetivo. Por eso cada batch queda asociado con el modelo, el momento de emisión, las estaciones cubiertas y los horizontes solicitados.

La métrica principal es una transformación de WAPE. Primero se calcula por estación:

```
WAPE = suma de errores absolutos / suma de valores reales
Accuracy = 100 × máximo(0, 1 - WAPE)
```

La accuracy oficial es el promedio no ponderado de las accuracies de las 12 estaciones. Así una estación de gran volumen no puede ocultar el mal desempeño en estaciones más pequeñas. Un target ausente se evalúa como predicción cero y queda reflejado también en la cobertura.

Cuándo aparece el resultado

Los targets se resuelven cuando su observación real ha sido liberada. Por eso un ciclo puede evaluarse de manera progresiva: algunos horizontes se conocen antes que otros. Después de cada lote, el sistema actualiza componentes de error y snapshots del leaderboard.

El leaderboard ofrece dos lecturas complementarias:

- **Acumulada:** resume toda la competencia desde su inicio.
- **Rolling 24 horas:** muestra el desempeño reciente y ayuda a identificar recuperación o degradación ante drift.

Tres señales distintas

Señal	Qué significa	Ejemplo de respuesta
Performance drift	El accuracy rolling 24 h cae	Revisar errores y comparar con el baseline
Data drift	Cambia la distribución de entradas	Examinar estaciones, horarios y variables afectadas
Falla operacional	El pipeline no produjo o envió resultados	Corregir la operación antes de culpar al modelo

El reentrenamiento no debe reaccionar a un único periodo difícil. La decisión considera persistencia, volumen de datos nuevos y tiempo transcurrido desde el último entrenamiento.

Automatizaciones esperadas

Collector

Se activa cada 30 minutos, recorre el stream desde el último cursor confirmado, valida la respuesta y actualiza Supabase mediante `upsert`. El cursor solo avanza después de confirmar la transacción. Debe dejar evidencia incluso cuando no encuentre novedades.

Entrenamiento

Puede activarse manualmente y también de forma programada o condicionada. Produce un candidato, lo compara y registra la decisión. Reentrenar no implica promover automáticamente.

Inferencia y submission

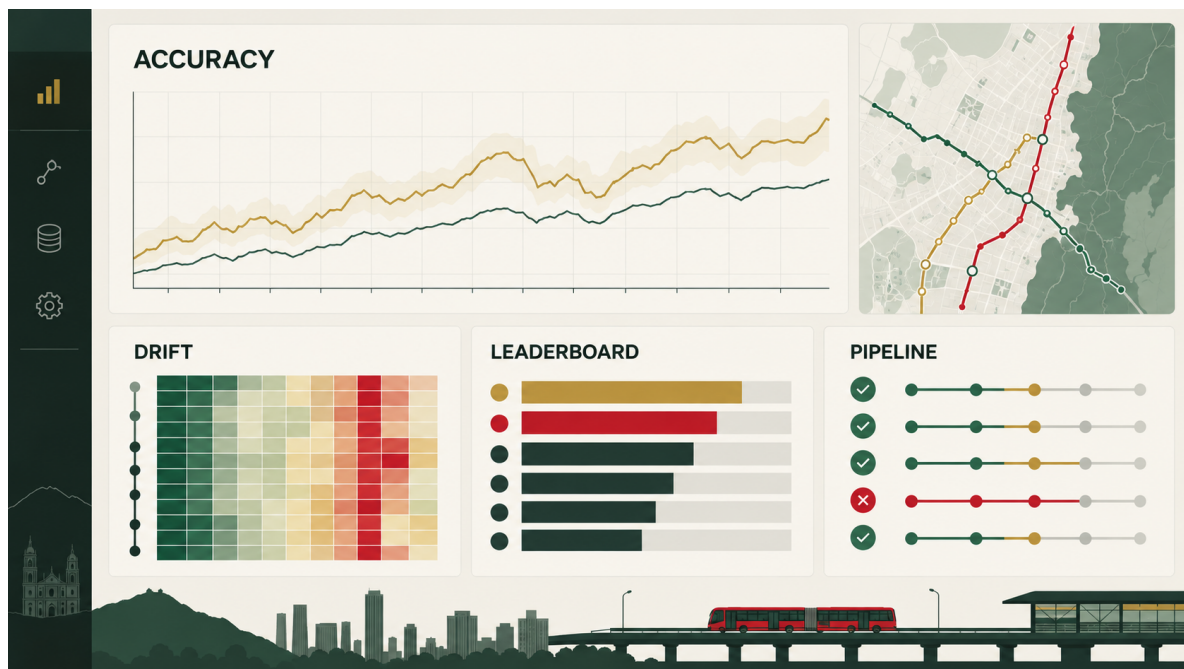
Consulta el ciclo vigente, carga el champion, construye las variables permitidas, genera los 48 valores y envía el batch antes del cierre. Un reintento conserva la misma llave de idempotencia y no debe crear submissions contradictorias.

Evaluación y monitoreo

Cuando aparece la realidad, une cada target con su predicción, calcula métricas locales, consulta el leaderboard y determina si hay una señal que merezca investigación o reentrenamiento.

GitHub Actions coordina el trabajo, pero la base conserva la memoria. Cada ejecución comienza en un entorno limpio; por eso el estado importante no puede depender del disco temporal del runner.

Bono — hacer visible el sistema



Concepto de dashboard opcional: desempeño, drift y operación visibles en una sola lectura.

Fuente: Concepto visual generado para la guía; no representa producto en operación, 2026-09-16

El dashboard en Vercel es un bono porque lleva el proyecto de “funciona” a “se puede observar y explicar”. No se premia por tener más gráficos, sino por revelar decisiones relevantes.

Una buena vista debería permitir

- reconocer el accuracy actual y su tendencia;
- comparar estaciones;
- ver cuál modelo es champion y desde cuándo;
- identificar drift sin leer logs;
- detectar ejecuciones fallidas;
- consultar la posición en el leaderboard;
- relacionar una caída de desempeño con un cambio de modelo o de datos.

El dashboard utiliza acceso de solo lectura. Las credenciales administrativas de Supabase y la API key de submissions nunca deben llegar al navegador.

Entregables

Núcleo obligatorio

1. Repositorio público, organizado y reproducible.
2. Descripción del problema y análisis exploratorio.
3. Supabase con histórico migrado y esquema documentado.
4. Collector incremental automatizado.
5. Baselines y experimentos con validación temporal.
6. Registro de versiones y modelo champion.
7. Inferencia periódica mediante GitHub Actions.
8. Predicciones almacenadas y submissions trazables.
9. Evaluación de accuracy a lo largo del tiempo.
10. Detección de degradación y decisión de reentrenamiento.
11. Informe final: qué cambió, qué funcionó y qué harían después.

Bonos

- Dashboard desplegado en Vercel.
- Experimentación y versionamiento con MLflow.
- Pruebas automatizadas y controles adicionales de calidad.
- Estrategia de rollback del modelo.
- Monitoreo de drift más allá de la métrica de desempeño.

Criterio de éxito

El mejor proyecto no necesariamente tendrá la mejor predicción en todos los ciclos. Será aquel que combine desempeño, trazabilidad, capacidad de recuperación y decisiones justificadas.

Ritmo sugerido de trabajo

Momento	Foco	Resultado observable
Semana actual	Comprensión y arquitectura	Hipótesis, repositorio y Supabase listos
Semana actual	Datos y modelado inicial	Histórico, collector probado, baselines y champion inicial
Próxima semana	Ventana competitiva	Predicciones horarias, evaluaciones y decisiones de reentrenamiento
Cierre	Síntesis	Evidencia reproducible y reflexión del equipo

Qué puede hacerse desde ahora

La API pública ya permite descargar 12 estaciones y 51.840 observaciones históricas. Los equipos pueden iniciar el análisis, migrar datos, diseñar el collector, ensayar el manejo de cursores y entrenar sus primeros modelos. El reloj seguirá en `waiting` y el stream vacío hasta la activación docente.

Qué se habilitará para la competencia

La infraestructura ya contiene ciclos, autenticación, submissions, deadlines, scoring y leaderboard. Al activar el escenario se crearán los participantes y se entregará una API key privada a cada uno. Los payloads y comandos exactos viven en el repositorio oficial para que esta guía conserve su función metodológica.

Próximos pasos

1. Crear el repositorio público del equipo y documentar su arquitectura.
2. Crear Supabase y migrar el histórico inicial.
3. Implementar y probar el collector idempotente aunque el stream aún esté vacío.
4. Ejecutar el baseline antes de diseñar modelos más complejos.
5. Preparar el workflow de inferencia para consultar el ciclo en lugar de asumir horarios.

Fuente técnica vigente

Repositorio público y contrato técnico: <https://github.com/uexternadojz/pulso-transmi>
API y documentación: <https://pulso-transmi.72-60-245-2.sslip.io/docs>

Evaluación conceptual propuesta

Dimensión	Peso orientativo
Comprensión y calidad de datos	15 %
Experimentación y validación temporal	20 %
Collector y persistencia	15 %
Versionamiento y promoción	10 %
Inferencia y submissions	15 %
Monitoreo, drift y reentrenamiento	15 %
Reproducibilidad y documentación	10 %

Los bonos pueden reconocer el dashboard, MLflow y capacidades adicionales sin reemplazar ningún componente obligatorio. La rúbrica definitiva se publicará junto con el contrato de competencia.

Límites de esta versión

- Esta es una guía metodológica; los payloads, secretos y comandos viven en el repositorio.
- La cadencia oficial es publicación cada 30 minutos, ciclo cada 60 y ventana de entrega de 25 minutos.
- Los workflows programados pueden presentar retrasos; cada ejecución debe consultar `closes_at` y nunca asumir que el cron ocurrió exactamente a la hora configurada.
- La señal de drift y los umbrales de reentrenamiento deben justificarse, no copiarse como una cifra universal.
- Vercel es una capa de visualización, no el motor periódico del proyecto.

Fuentes y corte

Documento preparado el 16 de septiembre de 2026 con base en el diseño vigente de Pulso TransMi y en las capacidades documentadas por las plataformas:

- Pulso TransMi: <https://github.com/uexternadojz/pulso-transmi>
- GitHub Actions, ejecución y programación: <https://docs.github.com/en/actions>
- Supabase Database y Storage: <https://supabase.com/docs>
- Vercel Hobby y Cron Jobs: <https://vercel.com/docs>
- MLflow Model Registry: <https://mlflow.org/docs/latest/ml/model-registry>

Las ilustraciones y diagramas son elaboración editorial para esta guía. El dashboard mostrado es conceptual y no representa una aplicación ya entregada.